Doc Query Django Project: The Complete Plain-English Guide

Oct 28, 2026 · @Aman Singh

Doc Query is a Django backend where users register, log in, save documents and log queries; this guide explains every file, idea and flow in simple words.

1. What this project is

Doc Query is the backend of an application: the engine behind the screen. It has no screen of its own. Other programs, such as Postman, talk to it using URLs and get answers back as JSON.

A simple way to picture it is a locker room. Registering gets you a locker, your token is the key, and documents are what you keep inside. Nobody else can open your locker.

What a user can do with it

- Register an account and log in to get a token.
- Save documents, either as plain text or as an uploaded file.
- List, open and delete their own documents.
- Send a query (a question) and have it saved together with a response.
- See their own query history.
- Log out, which makes the token stop working.

Four facts to remember

1. It is built with Django (the web framework) and Django REST Framework, called DRF (the toolkit for building APIs).
2. Data is stored in a database file called `db.sqlite3`, so it survives a server restart. Assignment 1 had no database; this one does.
3. Every user only sees their own documents and queries.
4. The answer to a query is a placeholder: the text `Processed: <your query>`. Real AI answers arrive in Assignment 3A, which builds on this project.

What the assignment asked for

The assignment says: build a persistent backend with full CRUD APIs, token-based authentication, and pagination. CRUD means Create, Read, Update, Delete. Here, documents can be created, read and deleted; there is no update (edit) endpoint.

2. The big picture: Django and DRF in plain words

Django is a Python web framework. It is called "batteries included" because it ships with a database layer, user accounts, an admin site, URL routing and security features. You write less code because much of the work is already done.

Django REST Framework (DRF) is an add-on for Django that makes building APIs easy. It gives you serializers, token authentication, pagination and ready-made view classes.

An API is a way for programs to talk to each other. One program sends a request, the other sends back data, usually JSON.

Project vs app. A Django project is the whole thing and its configuration. An app is one feature module inside it. Here the project is `doc_query` and it contains one app, `api`.

MVT. Django organises code as Model, View, Template. The model is the database part, the view holds the logic, and the template makes HTML pages. This project returns JSON, so it has no templates. DRF serializers decide what the client sees instead.

ORM. The Object-Relational Mapper lets you use Python classes instead of writing SQL. For example, `Document.objects.filter(owner=user)` becomes a SQL SELECT behind the scenes.

Migration. A migration is a set of instructions that creates or changes database tables. `makemigrations` writes the instructions from your models, and `migrate` runs them.

The whole journey in one line: URL, then view, then serializer, then model, then database, and the answer travels back the same way.

Assignment 1 versus Assignment 2

Idea	FastAPI (Assignment 1)	Django + DRF (Assignment 2)
Validate data	Pydantic models	Serializers
An endpoint	A function with a decorator	A view class
Routing	APIRouter, include_router	urls.py and path()
Database	None	Models, ORM, SQLite

Idea	FastAPI (Assignment 1)	Django + DRF (Assignment 2)
Authentication	X-API-Key header	Token in Authorization header
Run command	uvicorn	manage.py runserver
Auto docs page	/docs	Not by default

3. File structure: what every file does

The project is a folder with two parts: `doc_query` (the project settings) and `api` (the app where the real work happens).

```
Python_GenAI_Bootcamp_Assignment_2-main/
  manage.py
  requirements.txt
  README.md
  .gitignore
  db.sqlite3        (created by migrate)
  app.log           (created when it runs)
  documents/        (created on first file upload)
  doc_query/        (the project)
    __init__.py
    settings.py
    urls.py
    wsgi.py
    asgi.py
  api/              (the app)
    __init__.py
    models.py
    serializers.py
    views.py
    urls.py
    admin.py
    apps.py
    tests.py
    migrations/
      __init__.py
      0001_initial.py
```

Top level

File	What it does
`manage.py`	The command tool. You use it to run the server, migrate and create users.
`requirements.txt`	The packages to install: Django, DRF and three small helpers.
`README.md`	Documentation for humans.
`.gitignore`	Tells git what not to upload, such as the database and logs.
`db.sqlite3`	The database. All users, documents and queries live here.
`app.log`	The log file where events are written.
`documents/`	Where uploaded files are stored.

The `doc_query` folder (project settings)

File	What it does
`settings.py`	The control panel: installed apps, database, authentication, pagination, logging.
`urls.py`	The main address book. It sends `/admin/` to the admin site and `/api/` to the app.
`wsgi.py`	Entry point for traditional servers when you deploy.
`asgi.py`	Entry point for async servers when you deploy.
`__init__.py`	An empty file that marks the folder as a Python package.

The `api` folder (the app)

File	What it does
`models.py`	Defines the three database tables.

File	What it does
`serializers.py`	Converts model data to JSON and back, and validates input.
`views.py`	The logic for every endpoint. This is the biggest file.
`urls.py`	Maps each path, like `documents/`, to a view.
`admin.py`	Registers the models so they show up in the admin site.
`apps.py`	The app's own configuration. You rarely touch it.
`tests.py`	Meant for automated tests. It is empty right now.
`migrations/0001_initial.py`	Auto-generated recipe that creates the three tables.

The files you actually wrote and will be asked about are `models.py`, `serializers.py`, `views.py`, `urls.py` and the changes in `settings.py`.

4. The database: the three models

A model is a Python class that describes one database table. Each field in the class is a column, and each saved object is a row. Django adds an `id` column (the primary key) to every model automatically.

The file is `api/models.py`. The tables exist because `migrate` ran the recipe in `0001_initial.py`.

Document: one saved document

Field	Type	Meaning
`title`	CharField, max 255	Short text, required
`content`	TextField, can be blank	Long text body
`file`	FileField, optional	An uploaded file, stored under `documents/`
`owner`	ForeignKey to User	Who owns it
`created_at`	DateTimeField	Set once, when created

Field	Type	Meaning
`updated_at`	DateTimeField	Updated on every save

QueryLog: one saved query

Field	Type	Meaning
`user`	ForeignKey to User	Who asked
`query_text`	TextField	The question
`response_text`	TextField, can be blank	The answer (a placeholder for now)
`created_at`	DateTimeField	When it was asked

UserSession: one login session

Field	Type	Meaning
`user`	ForeignKey to User	Whose session
`session_key`	CharField, unique	A random id made at login
`started_at`	DateTimeField	When they logged in
`ended_at`	DateTimeField, optional	When they logged out
`is_active`	BooleanField	True until logout

How they connect. One user can have many documents, many queries and many sessions. This is a one-to-many relationship, made with `ForeignKey`.

The User model is not yours. Django already includes a User table with username, email and a hashed password. The models point to it using `settings.AUTH_USER_MODEL`. DRF also adds a Token table that links one token to one user.

Terms you should be able to explain

- `ForeignKey`: a column that points to a row in another table.
- `on_delete=CASCADE`: if the user is deleted, their documents, queries and sessions are deleted too.

- `related_name='documents'`: lets you write `user.documents.all()` to get a user's documents.
- `auto_now_add=True`: the date is filled once, when the row is created.
- `auto_now=True`: the date is refreshed every time the row is saved.
- `null=True`: the database may store an empty value. `blank=True`: forms and serializers may leave it empty. File and ended_at fields use both.
- `unique=True`: no two rows may have the same value.
- `upload_to='documents/'`: the folder where uploaded files are saved.
- `__str__`: the readable name shown in the admin site.

5. Serializers: data to JSON and back

A serializer is a translator and a gatekeeper. It turns a database object into JSON for the response, and it turns incoming JSON into a validated object for saving. It is the DRF version of a Pydantic model.

All four serializers are `ModelSerializer` classes. That means DRF builds the fields from the model for you. The inner `class Meta` says which model to use and which fields to include.

RegisterSerializer

- Fields: `id`, `username`, `email`, `password`.
- `password` is `write_only`: it can be sent in, but it is never sent back.
- `min_length=8` rejects short passwords.
- Its `create()` method calls `User.objects.create_user(...)`. That hashes the password, so the real password is never stored.

DocumentSerializer

- Fields: `id`, `title`, `content`, `file`, `owner`, `created_at`, `updated_at`.
- `owner`, `created_at` and `updated_at` are `read_only`. The client cannot set them.
- This matters for security. The owner is filled in by the view from the logged-in user, so nobody can create a document in someone else's name.
- `title` is required because the model field is not blank. A request without it gets a 400.

QueryLogSerializer

- Fields: `id`, `user`, `query_text`, `response_text`, `created_at`.
- `user`, `response_text` and `created_at` are read-only. The server decides them.

UserSessionSerializer

- Defined for the session model, but no view uses it yet.

How validation works

1. A view creates the serializer with the incoming data: `RegisterSerializer(data=request.data)`.
2. It calls `serializer.is_valid(raise_exception=True)`.
3. If the data is bad, DRF raises an error that normally becomes a 400 response listing what is wrong.
4. If it is good, `serializer.save()` writes to the database.

Two words to keep apart

- `read_only`: the field appears in responses but is ignored in requests.
- `write_only`: the field is accepted in requests but never appears in responses.

6. Views: the seven that do the work

A view receives a request, does the work and returns a response. All of them live in `api/views.py`. They come in two styles.

- **APIView:** you write the method yourself, such as `post()`. Used by Register, Login, Logout and Query.
- **Generic views:** DRF already wrote the standard behaviour. You only say which serializer and which data to use. Used by the document views and the query history.

Two settings inside each view

- `authentication_classes = [TokenAuthentication]`: how to find out who is calling (by the token).
- `permission_classes`: who is allowed in. `AllowAny` means anyone. `IsAuthenticated` means only someone with a valid token.

Inside a view, `request.data` is the parsed body and `request.user` is the logged-in user found from the token.

1. RegisterView (POST, anyone)

Validates the data with RegisterSerializer, creates the user, creates a token and returns 201 with `token`, `user_id` and `username`.

2. LoginView (POST, anyone)

If username or password is missing it returns 400. It checks them with Django's `authenticate()`; wrong ones return 401. On success it gets or creates the user's token, writes a UserSession row with a random key, and returns the token.

3. LogoutView (POST, token required)

Marks the user's active sessions as inactive and sets `ended_at`. Then it deletes the token, so that token can never be used again. It returns 200.

4. DocumentListCreateView (GET and POST, token required)

- `get_queryset()` returns only the current user's documents, newest first.
- `perform_create()` saves a new document with `owner` set to the current user.
- It accepts plain JSON or a file upload sent as form-data.

5. DocumentDetailView (GET and DELETE, token required)

- The queryset is filtered to the current user's documents. A document that belongs to someone else is simply not found, so the answer is 404.
- `perform_destroy()` deletes the stored file from disk first, then the database row. The response is 204 with no body.

6. QueryView (POST, token required)

Reads `query_text`; if it is missing or empty it returns 400. It builds the placeholder answer `Processed: <query_text>`, saves a QueryLog and returns it with 201. This is where a real LLM call will be added later.

7. QueryLogListView (GET, token required)

Returns the current user's saved queries, newest first, split into pages.

Logging. Every view writes to a logger: `info` for normal events, `warning` for bad input and `exception` for crashes. These lines appear in the terminal and in `app.log`.

Method names to know: `get_queryset` (which rows), `perform_create` (what happens on save) and `perform_destroy` (what happens on delete) are hooks that DRF calls for you.

7. URLs and endpoints

How routing works. Django looks up the address in two steps.

1. `doc_query/urls.py` is the main address book. It sends anything starting with `api/` to the app using `include('api.urls')`, and `admin/` to the admin site.
2. `api/urls.py` matches the rest of the path to a view.

So `documents/` inside `api/urls.py` becomes `/api/documents/` in the browser.

Things to know about the lines in `api/urls.py`:

- `path('documents/<int:pk>/', ...)`: `<int:pk>` captures a whole number from the URL and hands it to the view as `pk` (the primary key, the document's id).
- `.as_view()` turns a view class into something Django can call.
- Every path ends with a slash. Without it, Django will not match the request.

All endpoints

Method	Path	Token	Success
POST	`/api/register/`	No	201
POST	`/api/login/`	No	200
POST	`/api/logout/`	Yes	200
POST	`/api/documents/`	Yes	201
GET	`/api/documents/`	Yes	200
GET	`/api/documents/{id}/`	Yes	200
DELETE	`/api/documents/{id}/`	Yes	204
POST	`/api/query/`	Yes	201
GET	`/api/queries/`	Yes	200

There is also `/admin/`, Django's built-in admin site. You log in there with a superuser made by `python manage.py createsuperuser`.

What you send

- Register: JSON with `username`, `password` (at least 8 characters) and optional `email`.
- Login: JSON with `username` and `password`.

- Create document, as text: JSON with `title` and `content`.
- Create document, as a file: form-data with `title` (text) and `file` (a file).
- Query: JSON with `query_text`.
- Every protected call: the header `Authorization: Token <your token>`.

What list endpoints return. The list of documents and the query history both come back in the same wrapper:

```
{
  "count": 12,
  "next": "http://127.0.0.1:8000/api/documents/?page=2",
  "previous": null,
  "results": [ ... ]
}
```

`count` is the total number of items, `next` and `previous` are links to neighbouring pages (or null), and `results` is the current page.

Status codes in this project

Code	Meaning	Where it appears
200	OK	Login, logout, GET requests
201	Created	Register, new document, new query
204	Done, no body	Delete document
400	Bad input	Missing fields, empty query
401	Not authenticated	Bad login, missing or invalid token
404	Not found	Unknown id, or someone else's document
500	Server error	Unexpected crash

8. Authentication: how tokens work

Two words to keep apart. Authentication proves who you are. Permission decides what you are allowed to do once Django knows who you are.

What a token is. After you log in, the server gives you a long random string. It is saved in a database table next to your user. From then on you send that string with every request instead of your password.

The life of a token

1. You register or log in. The server returns `token`.
2. You send it on every protected call as the header `Authorization: Token <your token>`.
3. DRF's `TokenAuthentication` reads the header, looks the token up in the database and finds your user. That user becomes `request.user`.
4. The `IsAuthenticated` permission checks that a user was found. If not, the answer is 401.
5. You log out. The server deletes the token, so the same string now returns 401 "Invalid token".

Why 401 and not 403. TokenAuthentication tells DRF how to ask for credentials (a `WWW-Authenticate` header). When that exists, DRF uses 401 for "not authenticated". The 403 code is for "authenticated but not allowed".

Passwords. Django never stores the real password. `create_user` saves a hash, and `authenticate()` hashes what you typed and compares the two.

Global rules and per-view rules. `settings.py` sets TokenAuthentication and IsAuthenticated as the defaults for every view. Register and Login override this with `AllowAny`, because nobody has a token yet when they call them.

UserSession is separate from the token. The token proves identity. UserSession is a log you keep yourself: one row per login with a start time, an end time and an active flag.

Honest weak points (good to mention in the viva)

- The token never expires. It only stops working at logout.
- Each user has exactly one token. Logging in on two devices shares it, and logging out on one ends both.
- It needs HTTPS in real life, or the token can be read on the network.
- Production systems often use JWT or OAuth, which have expiry and scopes.

CSRF note. Django has CSRF protection for browser forms and the admin site. Token-authenticated API calls do not use cookies, so DRF does not require a CSRF token for them.

9. Settings that matter

`doc_query/settings.py` is the control panel of the whole project. Django reads it once at startup.

Basics

- `SECRET_KEY`: a secret string Django uses for signing and security. In this project it is written directly in the file, which is not safe. It should come from an environment variable.
- `DEBUG = True`: shows detailed error pages. Fine on your laptop, never in production.
- `ALLOWED_HOSTS = []`: the hostnames allowed to reach the server. Empty works for localhost while DEBUG is on.

INSTALLED_APPS. The list of apps Django loads. `rest_framework` turns on DRF. `rest_framework.authtoken` creates the Token table. `api` is your own app. If an app is not listed, its tables are not created.

MIDDLEWARE. A chain of small helpers that every request passes through on the way in, and every response on the way out. They handle security headers, sessions, CSRF, user lookup, messages and clickjacking protection.

ROOT_URLCONF. Points to `doc_query.urls`, where routing starts.

DATABASES. SQLite, stored in the file `db.sqlite3` in the project folder. It needs no installation, which is perfect for learning. Bigger projects use PostgreSQL or MySQL.

AUTH_PASSWORD_VALIDATORS. Rules like "not too common" and "not all digits". They apply to Django forms and `createsuperuser`. The register endpoint here only enforces the 8-character minimum from its serializer.

Time. `TIME_ZONE = 'UTC'` and `USE_TZ = True` mean times are stored in UTC with a timezone attached.

REST_FRAMEWORK. DRF's own settings block:

- Default authentication: TokenAuthentication.
- Default permission: IsAuthenticated, so everything is locked unless a view says otherwise.
- Pagination: `PageNumberPagination`, which splits lists into pages and reads `?page=2`.
- `PAGE_SIZE = 5`: five items per page. The assignment asks for 10, and the README also says 10.

LOGGING. Defines where log messages go.

- Two handlers: the console (your terminal) and a file, `app.log`.
- One format: level, time, logger name, message.
- The `api` logger records everything from DEBUG up. The `django` logger records WARNING and above.
- The levels, from least to most serious, are DEBUG, INFO, WARNING, ERROR, CRITICAL.
- Each view file does `logging.getLogger(__name__)`. Its name is `api.views`, which falls under the `api` logger.

Not set yet. `MEDIA_ROOT` and `MEDIA_URL` tell Django where uploaded files live and how to link to them. Without them, files land in a `documents/` folder in the project root.

10. One full request, step by step

Let us follow one real call: creating a document. Postman sends `POST http://127.0.0.1:8000/api/documents/` with the header `Authorization: Token abc123` and the body `{"title": "Notes", "content": "hello"}`.

1. The development server (`runserver`) receives the request.
2. The middleware chain runs: security, sessions, common checks, CSRF, user lookup.
3. Django opens `doc_query/urls.py`. The path starts with `api/`, so it passes the rest to `api/urls.py`.
4. In `api/urls.py`, `documents/` matches `DocumentListCreateView`.
5. DRF authenticates. `TokenAuthentication` reads the header, finds the token `abc123` in the database and sets `request.user` to its owner.
6. DRF checks permission. `IsAuthenticated` passes because a user was found. Without a valid token the request stops here with 401.
7. The method is POST, so the generic view runs its create logic. It builds `DocumentSerializer(data=request.data)`, and DRF's parser has already turned the JSON into a dictionary.
8. The serializer validates the data. The title is present, so it passes. If it failed, the request would stop here with 400.
9. `perform_create` runs `serializer.save(owner=request.user)`. Django's ORM writes a new row to SQLite and fills `created_at` and `updated_at`.
10. The logger writes `Document created: 'Notes' ...` to the terminal and to `app.log`.

11. The serializer turns the saved Document back into JSON: id, title, content, file, owner and the dates.
12. DRF sends it back as a response with status 201. It passes through the middleware and arrives in Postman.

The list call is almost the same. For `GET /api/documents/`, step 7 changes. `get_queryset()` selects only this user's documents, the paginator cuts out one page, and the serializer converts several documents at once into `results`.

Where a request can stop early

Problem	Stops at step	Result
Wrong path or missing trailing slash	3 or 4	404
No token or a bad token	5 and 6	401
Missing title or bad data	8	400
Unexpected crash in the code	any	500

11. How to run and test it

This project needs no `.env` file and no API keys.

Run it (Windows, VS Code)

1. Unzip the project. In VS Code choose File, then Open Folder, and pick the folder that contains `manage.py`. The zip has a folder inside a folder, so open the inner one.
2. Open the terminal with Ctrl+` and run these in order:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

3. Check that the prompt shows `(venv)` after the activate step.
4. You should see `Starting development server at http://127.0.0.1:8000/`. Leave that terminal running.

Commands worth knowing

Command	What it does
`python manage.py runserver`	Starts the server
`python manage.py makemigrations`	Writes migration files after you change models
`python manage.py migrate`	Applies migrations to the database
`python manage.py createsuperuser`	Makes an admin user for `/admin/`
`python manage.py shell`	Opens a Python prompt with Django loaded

Test in Postman

1. Import the file `DocQuery_Assignment2.postman_collection.json`.
2. Right-click the collection and choose Run collection. Untick folder 5, the manual file uploads.
3. The scripts save the token for you. If you test by hand instead, send `Authorization: Token <token>`. Do not use the Bearer option, because it sends the wrong word.
4. Every URL needs the trailing slash.

Requests tagged [SPEC] in that collection fail on purpose until the gaps in the next section are fixed.

If something goes wrong

Problem	Fix
`No module named django`	The venv is not active. Run `venv\Scripts\activate` and try again.
Every call returns 401	The header is wrong. Use `Authorization: Token <token>`.
404 on a correct path	You left off the trailing slash.
Errors after editing models	Run `makemigrations`, then `migrate`.
Django fails on Python 3.15	Install Python 3.12 and recreate the venv with `py -3.12 -m venv venv`.

Every new terminal needs the activate step again. Look for `(venv)` before you run anything.

12. Where the project does not match the assignment yet

I ran every endpoint against the assignment document. Most of it works, and six things do not match.

What already works

- Register and login return a token, and a wrong password gives 401.
- Calls with no token or a fake token give 401.
- Documents can be created from JSON and from a file, listed, opened and deleted (204).
- Users only see their own documents and queries.
- Query returns 201, and an empty query returns 400.
- Both lists come back as `count`, `next`, `previous`, `results`.
- After logout, the old token stops working.

What does not match

The assignment says	The project does
Fields named `uploaded_at`, `result_summary`, and a session with `token`, `created_at`, `expires_at`	Fields are `created_at`, `response_text`, and a session with `session_key`, `started_at`, `ended_at`, `is_active`
Allow only certain file types, and return 413 if a file is too big	No limits. A `.exe` and a 20 MB file were both accepted
Default page size 10, and a `page_size` option	Page size is 5, and `page_size` is ignored
Deleting someone else's document returns 403	It returns 404
Bad registration input should be a 400 error	A duplicate username or a short password returns 500
Paths like `/documents`	Paths are `/api/documents/`. Probably fine, but confirm with your trainer

Why register returns 500. The view wraps everything in `except Exception`. A validation error is also an Exception, so it gets caught and turned into a 500 instead of the 400 DRF would have sent.

Smaller things

- `SECRET_KEY` is written in `settings.py`. The organisation's rule is to keep secrets out of code.
- `MEDIA_ROOT` is not set, so uploads go into a loose `documents/` folder.
- `tests.py` is empty.
- The README says to install with `pip install django djangorestframework` and says 10 per page. Use `requirements.txt`, and the page size is really 5.

When you are back, tell me and we will fix these one at a time, with the code and the reason for each change.

13. Likely viva questions with short answers

Say the answers in your own words. These are the ideas to hit.

What is Django, and why DRF on top? Django is a full web framework with an ORM, user accounts, admin and routing. DRF adds the API tools: serializers, token auth, pagination and generic views.

What is MVT? Model, View, Template. The model is the data, the view is the logic, the template makes HTML. This project returns JSON, so serializers play the role the template would.

What is the difference between a project and an app? The project (`doc_query`) holds configuration. An app (`api`) is one feature module. One project can hold many apps.

What is the ORM? It lets me use Python classes and methods instead of SQL. `Document.objects.filter(owner=user)` becomes a SELECT query.

makemigrations versus migrate? `makemigrations` writes migration files from my model changes. `migrate` runs them against the database.

What is a serializer, and how does it differ from Pydantic? It converts model objects to JSON and validates incoming JSON. Pydantic does similar work in FastAPI, but a ModelSerializer builds its fields from a database model.

APIView versus generic views? With APIView I write the method myself. Generic views like ListCreateAPIView already contain the standard list and create behaviour, so I only set the serializer and the queryset.

How does each user only see their own data? `get_queryset()` filters by `request.user`. A document that is not theirs is not in that queryset, so the lookup returns 404.

How does token authentication work? Login returns a random token stored in the database. The client sends `Authorization: Token <key>`, DRF finds the user from it, and logout deletes it.

Why is the token better than sending the password every time? The password is exposed less often, and a token can be revoked by deleting it without changing the password.

Why are passwords hashed? So that a database leak does not reveal real passwords. `create_user` hashes, and `authenticate()` compares hashes.

Why is `owner` read-only in DocumentSerializer? The client must not choose the owner. The view sets it from the authenticated user with `serializer.save(owner=request.user)`.

What does `perform_create` do? It is the hook DRF calls after validation, just before saving. I use it to attach the owner.

What are ForeignKey and `on_delete=CASCADE`? A ForeignKey links a row to a row in another table. CASCADE means deleting the user also deletes their linked rows.

What is middleware? Code that every request and response passes through, such as security headers, sessions and authentication.

How does pagination work here? `PageNumberPagination` splits a list into pages. The client uses `?page=2`, and the response has `count`, `next`, `previous` and `results`. The page size comes from `PAGE_SIZE`.

How are uploaded files stored? The file goes to disk under `upload_to='documents/'`. The database only stores the file's path.

AllowAny versus IsAuthenticated? AllowAny lets anyone in, used for register and login. IsAuthenticated needs a valid token, and it is the default for everything else.

Why SQLite, and when would you change it? It needs no setup and is good for learning. For many users or heavy writes I would move to PostgreSQL.

What does logging give you? A record of what happened, with levels from DEBUG to CRITICAL. Here it goes to the console and to `app.log`.

What are the weaknesses of this project? Tokens never expire, file types and sizes are not checked, `SECRET_KEY` is in the code, the page size and some field names differ from the assignment, there are no tests, and the query answer is only a placeholder.

Why does register return 500 for a duplicate username? The view catches every Exception, including validation errors. The fix is to let DRF's validation error return its 400, and keep the catch-all only for real crashes.

How will this grow in Assignment 3A? Documents will be split into chunks and embedded into a vector store. A query will retrieve the most relevant chunks and send them to an LLM, and that answer will replace the `Processed:` placeholder.

14. Quick glossary

Term	Meaning
API	A way for programs to talk to each other by sending requests and getting data back.
APIView	A DRF view class where you write the HTTP methods yourself.
App	One feature module inside a Django project. Here: `api`.
Authentication	Proving who you are.
Permission	Deciding what an identified user may do.
CRUD	Create, Read, Update, Delete.
CSRF	An attack on browser sessions. Django guards forms and admin against it.
DRF	Django REST Framework, the API toolkit built on Django.
Endpoint	One URL plus one HTTP method that does one job.
FileField	A model field that stores an uploaded file and saves its path.
ForeignKey	A column linking a row to a row in another table.
Generic view	A DRF view with standard behaviour built in.
Hash	A one-way scrambled form of a password. It cannot be turned back.

Term	Meaning
Header	Extra information on a request, such as `Authorization`.
JSON	A text format for data using keys and values.
manage.py	Django's command tool.
Middleware	Code every request and response passes through.
Migration	A recipe that creates or changes database tables.
Model	A Python class that describes a database table.
MVT	Model, View, Template: Django's way of organising code.
ORM	Lets you use Python instead of SQL to work with the database.
Pagination	Splitting a long list into pages.
Primary key (`pk`)	The unique id of a row.
Project	The whole Django setup and its settings. Here: `doc_query`.
Queryset	A set of database rows returned by the ORM.
Request	What the client sends to the server.
Response	What the server sends back.
Serializer	Converts objects to JSON and validates incoming JSON.
SQLite	A simple database stored in one file.
Status code	A number saying how the request went, such as 200 or 404.
Token	A random string that identifies a logged-in user.
URL routing	Matching a web address to the view that handles it.
View	Code that receives a request and returns a response.

Term	Meaning
Virtual environment	A private folder of Python packages for one project.

When you are back, we will run the project, test every endpoint in Postman and then fix the gaps from section 12.